

STAR - OIDS GDD

STAR-OIDS es un juego 2D de disparos estilo Asteroids ambientado en el universo de Star Wars: el jugador pilota un X-Wing o un Tie Fighter, dispara a asteroides que se fragmentan al ser destruidos, y debe sobrevivir el mayor tiempo posible acumulando puntos, con un multiplicador que crece cada minuto de partida. El proyecto se ha desarrollado en dos iteraciones (mecánicas base y mecánicas avanzadas) más un conjunto de extras (sonido, menú de configuración, selección de nave, HUD). Se usa Unity 6 2D y C# como lenguaje.

1. ESCENAS

MainMenu: menú principal con navegación entre paneles: Jugar, Selección de nave (X-Wing / Tie Fighter), Configuración (volumen, pantalla completa, mostrar/ocultar HUD) y Créditos. Reproduce la música del menú en bucle desde que se carga la escena.

SampleScene: escena de juego con cámara ortográfica 2D que sigue al jugador (CameraFollow.cs) con un offset fijo. También tiene al jugador, el spawner de asteroides, el HUD (vidas, puntuación), y los paneles de Pausa, Configuración y Game Over, todos desactivados en la jerarquía hasta que el código los activa en el momento que ya les toca.

2. ASSETS, GAME OBJECTS, UI, AUDIO

ASSETS & GAME OBJECTS

Nave del Jugador: dos variantes, X-Wing y Tie Fighter, cada una con su propio sprite, collider y arma (Gun.cs) asignados de diferentes formas.

Bala: un prefab por tipo de nave, reutilizado en tiempo real mediante Object Pooling en vez de instanciarse en cada disparo.

Asteroide grande, mediano y pequeño: tres prefabs encadenados: cada uno referencia (vía el enum NextSize) el tamaño que debe generar al fragmentarse, hasta llegar al pequeño, que ya no genera nada más.

Game Manager : para invocar varios scripts de pause menu, game over y lives manager.

LivesManager : vidas actuales del jugador.

GameOverManager: pantalla y lógica de fin de partida.

ScoreMultiplier : el multiplicador de puntuación vigente.

EnemySpawner: generación periódica de asteroides grandes.

Music Player: la que se encarga de poner la música del nivel.

BulletPool: el pool de balas para el caza tie porque solo dispara de 1 en 1.

BulletPoolSingle: el pool de balas para el x wing porque solo dispara de 4 en 4.

AsteroidPoolManager: los tres pools de asteroides (grande / mediano / pequeño).

Asteroid Wall: un tilemap con fondo de asteroides para poner límites del mapa.

Background: un tilemap con un fondo monótono para generar 1000 assets.

UI (INTERFAZ GRÁFICA)

HUD: vidas y puntuación en tiempo real, con una imagen que acompaña siempre a la cámara como un canvas anclado al jugador.

Panel de Pausa: accesible con ESC, navegable con teclado (Continuar, Reiniciar, Configuración, Salir).

Panel de Configuración: cada uno distinto entre MainMenu y SampleScene, con volumen y pantalla completa, son divididas para que si quieres acceder a config jugando no se resetee la partida.

Panel de Game Over: puntuación final, opción de reiniciar o volver al menú principal.

Audio (Importado mayormente desde vídeos de youtube)

Música del menú principal, en bucle.

Música de partida, con temática Star Wars, en bucle.

Sonido de disparo del Tie Fighter y del X-Wing (distintos entre sí).

Sonido de destrucción de asteroide.

Sonido de pérdida de vida, estilo Pacman.

Sonido de Game Over, que detiene la música de partida.

3. MECÁNICAS

Iteración 1 – mecánicas base (60%)

- **Escena 2D + cámara:** cámara ortográfica configurada para el juego 2D, con seguimiento dinámico del jugador.
- **Movimiento y rotación:** impulso físico mediante `Rigidbody2D.AddForce` con velocidad máxima limitada, y rotación libre controlada con las teclas.
- **Disparo:** arma con tiempo de recarga propio de cada nave, controlado comparando `Time.time` en vez de coroutines para que sea más preciso y tal.
- **Aparición de enemigos:** asteroides grandes generados periódicamente en los bordes de la cámara (recalculados cada vez, porque la cámara se mueve), dirigidos hacia el jugador.
- **Puntuación (UI Scoring):** HUD con vidas y puntuación actualizadas en tiempo real desde `ScoreManager`.

Iteración 2 – mecánicas avanzadas (40%)

- **Fragmentación de asteroides:** un asteroide grande se parte en 2 medianos, y cada mediano en 4 pequeños, repartidos simétricamente alrededor de la dirección de impacto mediante una rotación de vector 2D calculada a mano.
- **Pausa y reinicio:** la tecla ESC congela el tiempo global (`Time.timeScale = 0`) y muestra un panel navegable por teclado gracias al `EventSystem` de Unity.
- **Object Pooling:** tanto las balas como los tres tamaños de asteroides se reciclan (`SetActive true/false`) en vez de crearse y destruirse cada vez, evitando el coste de `Instantiate/Destroy` repetido durante la partida.

Extras añadidos

- **Multiplicador de puntuación:** se duplica cada 60 segundos de partida, premiando las partidas largas.
- **Invulnerabilidad temporal:** tras recibir un impacto, el jugador queda unos segundos invulnerable con un parpadeo visual, evitando perder varias vidas de golpe.
- **Selección de nave:** elección persistente entre X-Wing y Tie Fighter, guardada en `PlayerPrefs` y leída por `Player.cs` al empezar cada partida.
- **Sonido:** música de menú y de partida, y efectos para disparo (por nave), destrucción de asteroide, pérdida de vida y Game Over.
- **Menú de configuración:** volumen, pantalla completa y mostrar/ocultar HUD, todo persistente entre sesiones.

Extras descartados (demasiadas fumadas)

- **Power-ups:** demasiado trabajo para una minientrega.
- **Animaciones:** demasiado trabajo para una minientrega.
- **Build para móvil:** descartado, no hace falta.
- **Cambio de teclas / idioma:** Demasiado trabajo y esfuerzo para este trabajo inicial.
- **Efecto visual CRT:** aunque sepa de Unity no sé tanto de shaders o de estas cosas.

4. MECÁNICAS

Resumen de los scripts principales del proyecto, para que sirva y demás:

SCRIPT	USO	USO DETALLADO
Player.cs	Naves	Movimiento con impulso físico de addforce + rotación, leer la nave elegida, invulnerabilidad
Gun.cs	Disparo	Dispara con cooldown, pide balas al object pool, hace sonido y las activa/desactiva en pool
Bullet.cs	Pooling	Controla la vida útil de la bala , detecta colisión y devuelve la bala al pool, reutilizando
ObjectPool.cs	Object Pool genérico	Para bullet, instancia, activa y desactiva las balas
Asteroid.cs	Fragmentación	Movimiento de línea recta, fragmentación en trozos en ángulos simétricos, suma de punto y sonido destrucción
AsteroidPoolManager.cs	Pooling	Gestiona pools independientes para grande, mediano y pequeño
EnemySpawner.cs	Spawner	Genera asteroides aleatoriamente según la cámara
ScoreManager.cs	HUD	Para ver tu puntuación
ScoreMultiplier.cs	Progresión	Multiplicador
GameOverManager.cs	HUD	Menú de muerte
PauseMenu.cs	HUD	Menú de pausa
MainMenuController.cs	HUD	Menú inicio
LivesManager.cs	HUD	Menú de vidas
ShipsMenuControllers.cs	HUD	Menú de selección
ConfigManu	HUD	Menú de configuración
CameraFollow	Cámara	Cámara de seguimiento
MapBorder	Bordes	Para los bordes

5. INTERACCIÓN

Resumen de las interacciones:

ENTRADA	ACCIÓN
Flechas/WASD	Mover la nave
Barra espaciadora/click ratón	Disparar
ESC	Pausar/Renaudar la partida
Ratón	Navegar por los menús